Functions - Topic Explanation

2026-01-12

1. What is a Function?

When writing code, we often need to repeat the same operations many times (for example calculating GC content again and again).

Instead of writing the same code every time, we can place that code inside a **function**.

This means:

- We write the code once
- We call it whenever we need it

You can think of a function like a **kitchen blender**:

- **Input:** You put fruits into the blender.
- **Process:** The blender mixes the fruits.
- **Output:** It gives you fruit juice.

In programming, a function is like a small box where you give **input data**, it **processes the data**, and then returns a **result**.

---

2. Structure of a Function

General template:

```
function_name <- function(parameter) {
  # Operations to perform
  return(result)  # Final output
}
```

- `function_name`: the name we give to the function (for example `calculate_gc_content`)
- `parameter`: the input that the function receives
- `return(result)`: the final result that the function sends back

  Note: In R, `return()` is not always required.
  The last line is automatically returned.
  However, for beginners it is often clearer to use `return()`.

3. A Simple Example

```
multiply_by_two <- function(x) {
  result <- x * 2
  return(result)
}

multiply_by_two(5)
```

```
## [1] 10
```

Explanation

- `multiply_by_two` is the name of the function.
- `x` is the input of the function.
- `x * 2` multiplies the input by 2.
- The result is stored in the variable `result`.
- The result is returned by the function.
- To run the function, write its name and give a value inside parentheses: `multiply_by_two(5)`.

4. Function + Biological Example: GC Content

Goal: Write a function that calculates the GC content of a DNA sequence.

The function expects the DNA sequence in this format:

```
c("A","G","C","T")
```

```
## [1] "A" "G" "C" "T"
```

Writing the Function

```
calculate_gc_content <- function(dna) {

  gc_total <- 0

  for (n in dna) {
    if (n == "G" || n == "C") {
      gc_total <- gc_total + 1
    }
  }

  ratio <- gc_total / length(dna)
  return(ratio)
```

```
}

dna1 <- c("A", "G", "C", "T", "G")
calculate_gc_content(dna1)
```

```
## [1] 0.6
```

Understanding the GC Content Function Step by Step

- `calculate_gc_content` → the name of the function
- `function(dna)` → the function receives an input
- `dna` → the DNA sequence given to the function
- `{ }` → the block of code that runs inside the function
- `gc_total <- 0`
  - starts the GC counter
  - initial value is 0
- `for (n in dna)`
  - goes through each element of the DNA sequence
  - `n` is the current nucleotide
- `if (n == "G" || n == "C")`
  - checks if the nucleotide is G or C
  - `==` means equality check
  - `||` means OR
- `gc_total <- gc_total + 1`
  - increases the counter whenever we see G or C
- `ratio <- gc_total / length(dna)`
  - GC content = (number of G + C) / (total number of bases)
- `return(ratio)`
  - returns the final result

---

5. Bioinformatics Example: Leaf Length Check

Goal: Write a function that returns **"Long leaf"** if the leaf length is greater than 5, otherwise **"Short leaf"**.

```
leaf_analysis <- function(length) {
  if (length > 5) {
    result <- "Long leaf"
  } else {
    result <- "Short leaf"
  }
  return(result)
}

leaf_analysis(3.2)
```

```
## [1] "Short leaf"
```

```
leaf_analysis(6.1)
```

```
## [1] "Long leaf"
```